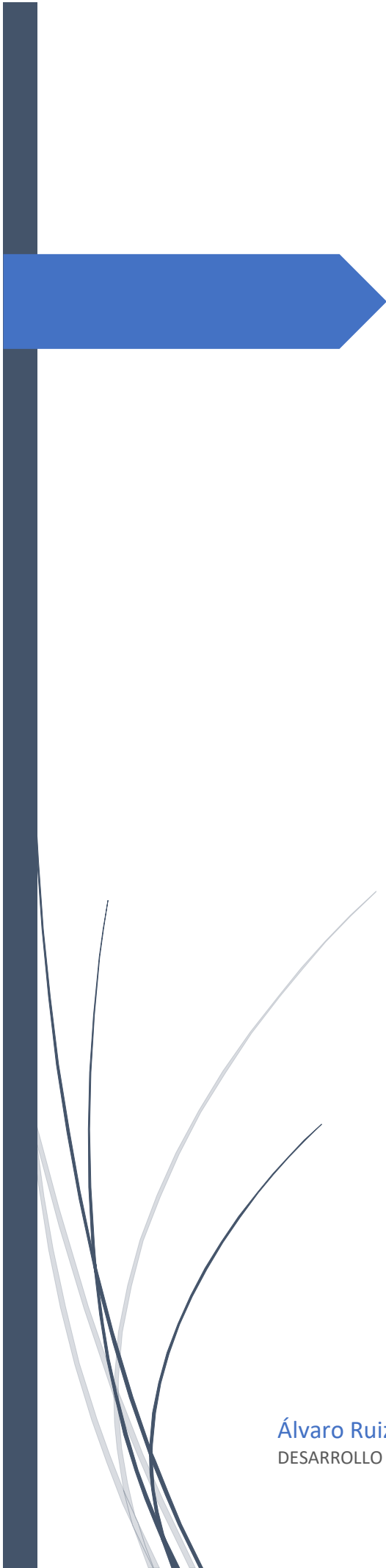




DWES-04- RepositorioRemoto

Álvaro Ruiz Velázquez // Adrián García-Heras
DESARROLLO WEB EN ENTORNOS SERVIDOR

Índice

1. Introducción.....	4
1.1. Descripción general del proyecto	4
1.2. Objetivos del proyecto.....	4
2. Descripción y funcionamiento de la aplicación.....	4
2.1. Funcionamiento general	4
2.2. Menú principal.....	5
2.3. Gestión de usuarios.....	5
2.4. Consumo de la API REST	5
2.5. Persistencia local	5
2.6. Sistema de caché.....	5
2.7. Sincronización automática.....	5
2.8. Exportación de datos	6
2.9. Sistema de notificaciones	6
3. Arquitectura del proyecto	6
3.1. Arquitectura general	6
3.2. Estructura de directorios.....	7
3.3. Interfaces y desacoplamiento.....	7
4. Modelos, DTO y mapeo de datos	8
4.1. Modelo de dominio User	8
4.2. Modelos relacionados: Address, Geo y Company.....	8
4.3. DTO utilizados por la API	8
5. Persistencia de datos.....	8
5.1. Repositorio de usuarios	8
5.2. Entity Framework Core	9
5.3. Estructura y almacenamiento de los datos.....	9
6. Sistema de caché.....	9
6.1. Funcionamiento de la caché	9

6.2. IUserCache y UserCache.....	9
6.3. Caché en desarrollo: MemoryCache.....	9
6.4. Caché en producción: Redis	9
6.5. Estrategia de consulta y actualización de la caché	10
7. Consumo de la API REST	10
7.1. JSONPlaceholder.....	10
7.2. Cliente Refit.....	10
7.3. Flujo de datos entre API, caché y base de datos	10
8. Servicios y lógica de negocio	11
8.1. UserService.....	11
8.2. Operación de consulta de usuarios	11
8.3. Creación de usuarios.....	11
8.4. Actualización de usuarios	11
8.5. Eliminación de usuarios	11
8.6. Exportación de usuarios.....	11
9. Validación y gestión de errores.....	12
9.1. Validación de usuarios	12
9.2. DomainErrors	12
9.3. Uso de Result/Result<T>	12
9.4. Tratamiento de errores	12
10. Sincronización y notificaciones.....	12
10.1. Sincronización automática.....	12
10.2. Funcionamiento de SynchroService	13
10.3. PeriodicTimer	13
10.4. Notificaciones.....	13
11. Exportación y almacenamiento JSON	13
11.1. UserStorage.....	13
11.2. Serialización con System.Text.Json.....	13
11.3. Generación del archivo JSON	14
12. Logging.....	14
12.1. Uso de Serilog	14
12.2. Organización de los mensajes.....	14

13. Contenerización con Docker	14
13.1. Objetivo de la contenerización	14
13.2. Docker Compose	14
13.3. PostgreSQL y Redis.....	15
13.4. Volúmenes y Healthchecks	15
14. Pruebas	15
14.1. Proyecto de pruebas.....	15
14.2. NUnit, Moq y FluentAssertions	15
14.3. Resultados obtenidos	15
15. Decisiones de diseño	16
15.1. Uso de interfaces	16
15.2. Separación de responsabilidades	16
15.3. Elección de SQLite y PostgreSQL	16
15.4. Elección de MemoryCache y Redis	16
15.5. Uso de DTO y Mappers.....	16
15.6. Uso de Refit.....	16
15.7. Uso de Docker	17
16. Diagramas y flujos de funcionamiento.....	17
16.1. Diagrama de arquitectura.....	17
16.2. Flujo de obtención de usuarios	17
16.3. Flujo de consulta por ID	18
16.4. Flujo de creación	18
16.5. Flujo de actualización.....	19
16.6. Flujo de eliminación	19
16.7. Flujo de sincronización.....	20

1. Introducción

1.1. Descripción general del proyecto

Este proyecto es una solución desarrollada en .NET 10 / C# diseñada para gestionar de forma eficiente la información de usuarios integrando múltiples fuentes de datos. La aplicación implementa una arquitectura orientada a servicios que permite consultar, crear, actualizar, eliminar y sincronizar datos provenientes de un servicio externo REST, apoyándose en estrategias de almacenamiento local, almacenamiento distribuido y almacenamiento en memoria para maximizar el rendimiento.

1.2. Objetivos del proyecto

- ✚ Implementar una Arquitectura Limpia con separación clara de responsabilidades.
- ✚ Integrar persistencia híbrida soportando SQLite para entornos de desarrollo/ligeros y PostgreSQL para entornos de producción.
- ✚ Diseñar un sistema de Caché multinivel compatible con IMemoryCache (local) e IDistributedCache (Redis).
- ✚ Consumir la API REST de JSONPlaceholder utilizando el cliente declarativo Refit.
- ✚ Desarrollar un servicio automático de sincronización en segundo plano con PeriodicTimer.
- ✚ Contenerizar la aplicación completa e infraestructura auxiliar mediante Docker y Docker Compose.
- ✚ Cubrir la lógica de negocio y casos de uso con pruebas unitarias exhaustivas usando NUnit, Moq y FluentAssertions.

2. Descripción y funcionamiento de la aplicación

2.1. Funcionamiento general

La aplicación opera como un servicio centralizado de gestión de usuarios. Ante una solicitud de consulta, evalúa primero la presencia del recurso en el sistema de caché; si no existe, consulta la base de datos local y, si aplica, sincroniza los datos. Las operaciones de escritura modifican la base de datos local y refrescan o invalidan la caché correspondientemente.

2.2. Menú principal

La aplicación dispone de una interfaz/console desde la cual el operador puede invocar manualmente la consulta de usuarios, el alta de nuevos registros, la actualización de datos existentes, el borrado físico y la exportación de datos en formato JSON.

2.3. Gestión de usuarios

Permite la realización de operaciones CRUD completas sobre la entidad User (incluyendo datos anidados de dirección Address, coordenadas Geo y empresa Company), aplicando validaciones estrictas antes de persistir cualquier cambio.

2.4. Consumo de la API REST

Mediante una interfaz gestionada por Refit, la aplicación realiza peticiones HTTP a la API externa de JSONPlaceholder para obtener el listado de usuarios originales y alimentar la base de datos local.

2.5. Persistencia local

Todos los usuarios se almacenan en una base de datos relacional local (SQLite o PostgreSQL). Esto garantiza la soberanía de los datos locales y la continuidad operativa aunque la API de JSONPlaceholder sufra caídas o latencia.

2.6. Sistema de caché

Para optimizar las lecturas frecuentes de usuarios por ID, la aplicación implementa la abstracción IUserCache. Si se habilita Redis, la caché se comparte entre instancias; de lo contrario, se utiliza la memoria RAM del proceso.

2.7. Sincronización automática

El servicio SynchroService ejecuta tareas periódicas en segundo plano que limpian la caché, vacían la base de datos local y vuelven a cargar los datos desde la API externa para mantener la coherencia absoluta del repositorio con el origen de datos primario.

2.8. Exportación de datos

Incluye la capacidad de volcar la totalidad de usuarios a archivos JSON estructurados en el sistema de archivos local a través del servicio UserStorage.

2.9. Sistema de notificaciones

El componente NotificationService captura eventos clave dentro de la aplicación y emite notificaciones operativas registradas a través del sistema de logs o servicios de mensajería.

3. Arquitectura del proyecto

3.1. Arquitectura general

El proyecto adopta un enfoque modular basado en Clean Architecture. Las capas se organizan de forma concéntrica o unidireccional: la capa de dominio no posee dependencias externas; los servicios coordinan los flujos de negocio; y los repositorios/controladores implementan los detalles técnicos de infraestructura (EF Core, Redis, Refit).

3.2. Estructura de directorios

```
RepositorioRemoto/  
├── Api/  
│   └── IJsonPlaceholderApi.cs  
├── Cache/  
│   ├── IUserCache.cs  
│   └── UserCache.cs  
├── Config/  
│   └── AppConfig.cs  
├── Dto/  
│   ├── Users/  
│   │   ├── AddressDto.cs  
│   │   ├── CompanyDto.cs  
│   │   ├── GeoDto.cs  
│   │   └── UserDto.cs  
│   ├── CreateUserRequest.cs  
│   └── UpdateUserRequest.cs  
├── Entities/  
│   ├── AppDb/  
│   │   ├── AppDbContextPostgres.cs  
│   │   ├── AppDbContextSqlite.cs  
│   │   └── IAppDbContext.cs  
│   └── Enums/  
│       └── TypeNotification.cs  
├── Errors/  
│   ├── ApiErrors.cs  
│   ├── DomainErrors.cs  
│   ├── RepositoryErrors.cs  
│   ├── ServiceErrors.cs  
│   ├── StorageErrors.cs  
│   └── UserErrors.cs  
├── Infrastructure/  
│   └── InyectorDependencias.cs  
├── Mappers/  
│   ├── AddressMapper.cs  
│   ├── CompanyMapper.cs  
│   └── UsuarioMapper.cs  
├── Models/  
│   ├── Address.cs  
│   ├── Company.cs  
│   ├── Geo.cs  
│   ├── Notification.cs  
│   └── User.cs  
├── Repository/  
│   ├── IUserRepository.cs  
│   └── UserRepository.cs  
├── Service/  
│   ├── Notifications/  
│   │   ├── INotificationService.cs  
│   │   └── NotificationService.cs  
│   ├── Synchro/  
│   │   ├── ISynchroService.cs  
│   │   └── SynchroService.cs  
│   └── Users/  
│       ├── IUserService.cs  
│       └── UserService.cs  
├── Storage/  
│   ├── IUserStorage.cs  
│   └── UserStorage.cs  
├── Validation/  
│   ├── IUserValidator.cs  
│   └── UserValidator.cs  
├── appsettings.dev.json  
├── appsettings.prod.json  
├── Dockerfile  
├── Program.cs  
└── RepositorioRemoto.csproj
```

3.3. Interfaces y desacoplamiento

Cada servicio o repositorio expone una interfaz (IUserService, IUserRepository, IUserCache, IJsonPlaceholderApi). Esto permite reemplazar fácilmente la infraestructura (por ejemplo, cambiar de SQLite a PostgreSQL o de MemoryCache a Redis) sin modificar

una sola línea de la lógica de negocio, además de facilitar la inyección de mocks en las pruebas unitarias.

4. Modelos, DTO y mapeo de datos

4.1. Modelo de dominio User

Es la entidad principal del sistema. Representa al usuario internamente y contiene las propiedades: Id, Name, Username, Email, Phone, Website, además de las relaciones complejas con Address y Company.

4.2. Modelos relacionados: Address, Geo y Company

- ✚ Address: Almacena la calle (Street), suite (Suite), ciudad (City), código postal (Zipcode) y la entidad anidada Geo.
- ✚ Geo: Almacena las coordenadas geográficas de latitud (Lat) y longitud (Lng).
- ✚ Company: Guarda el nombre de la empresa (Name), eslogan (CatchPhrase) y modelo de negocio (Bs).

4.3. DTO utilizados por la API

Los Data Transfer Objects definen el contrato exacto de serialización/deserialización HTTP con la API de JSONPlaceholder. Aíslan el modelo de base de datos interno de posibles cambios en el API externo.

5. Persistencia de datos

5.1. Repositorio de usuarios

Implementa el patrón Repository sobre IUserRepository. Abstrae las operaciones de acceso a datos (GetAllAsync, GetByIdAsync, AddAsync, AddRangeAsync, UpdateAsync, DeleteAsync, DeleteAllAsync) ofreciendo una interfaz limpia a la capa de servicios.

5.2. Entity Framework Core

Se utiliza EF Core como ORM. Se configuran las entidades mediante el método `OnModelCreating` o anotaciones para definir claves primarias, claves foráneas y comportamientos en cascada de los objetos anidados (Address, Company, Geo).

5.3. Estructura y almacenamiento de los datos

La arquitectura permite alternar entre `AppDbContextSqlite` y `AppDbContextPostgre` según el entorno de ejecución mediante variables de entorno o argumentos en consola al iniciar el programa, comprobando con la `AppConfig` (`appsettings.dev.json` y `appsettings.prod.json`).

6. Sistema de caché

6.1. Funcionamiento de la caché

El sistema de caché reduce la carga sobre la base de datos almacenando temporalmente en memoria los objetos User solicitados con mayor frecuencia.

6.2. IUserCache y UserCache

La interfaz `IUserCache` define los métodos del contrato de caché (`Get`, `Set`, `Remove`, `Clear`). La clase `UserCache` implementa esta interfaz encapsulando la abstracción de caché subyacente.

6.3. Caché en desarrollo: MemoryCache

Cuando no existe un servidor Redis configurado, se registra una implementación en memoria local, permitiendo un alto rendimiento dentro del propio proceso de la aplicación.

6.4. Caché en producción: Redis

En el entorno contenerizado con Docker Compose, se despliega una instancia de Redis. `UserCache` utiliza `IDistributedCache` serializando los objetos en JSON, permitiendo compartir el estado de la caché entre múltiples réplicas del servicio.

6.5. Estrategia de consulta y actualización de la caché

- ✚ Lectura (GetByIdAsync): Busca en caché. Si existe, lo devuelve. Si no, consulta la BBDD, guarda en caché y devuelve el resultado.
- ✚ Escritura/Actualización: Actualiza la BBDD y actualiza la entrada en la caché.
- ✚ Eliminación: Elimina de la BBDD y borra la entrada de la caché.
- ✚ Sincronización: Ejecuta un Clear() global de la caché para evitar inconsistencias con datos obsoletos.

7. Consumo de la API REST

7.1. JSONPlaceholder

Se utiliza la API pública accesible en [<https://jsonplaceholder.typicode.com/users>] como fuente externa original de datos de usuarios.

7.2. Cliente Refit

En lugar de escribir código cliente con HttpClient manual, se define la interfaz IJsonPlaceholderApi utilizando anotaciones de Refit. Refit genera automáticamente la implementación cliente.

7.3. Flujo de datos entre API, caché y base de datos

1. La API externa entrega DTOs.
2. SynchroService procesa estos DTOs, los convierte en entidades y los persiste en la BBDD local.
3. Las consultas posteriores leen de la BBDD local y alimentan la caché para acelerar accesos sucesivos.

8. Servicios y lógica de negocio

8.1. UserService

Es el servicio principal que coordina la lógica de negocio de usuarios. Hace de orquestador entre el repositorio, la caché, el validador, el mapper y el servicio de notificaciones.

8.2. Operación de consulta de usuarios

- ✚ GetAllUsersAsync: Recupera todos los usuarios del repositorio y los retorna como lista de UserDto.
- ✚ GetUserByIdAsync: Implementa la estrategia Cache-Aside descrita anteriormente.

8.3. Creación de usuarios

Valida los datos recibidos mediante UserValidator. Si es válido, convierte el DTO a Entidad, lo guarda en la base de datos, lo almacena en caché y emite una notificación de creación.

8.4. Actualización de usuarios

Comprueba la existencia previa del usuario. Tras validar la solicitud, aplica las modificaciones en el repositorio, actualiza la entrada en la caché y notifica la actualización.

8.5. Eliminación de usuarios

Verifica la existencia del recurso. Si existe, lo elimina de la base de datos, remueve su clave de la caché y emite la notificación correspondiente.

8.6. Exportación de usuarios

Obtiene los usuarios actuales y solicita a UserStorage la generación física del archivo JSON en el disco.

9. Validación y gestión de errores

9.1. Validación de usuarios

Utiliza la clase `UserValidator` para verificar reglas de negocio sobre los datos recibidos (formato de correo electrónico válido, campos obligatorios no vacíos como `Name` y `Username`).

9.2. DomainErrors

Define de forma explícita los errores conocidos del dominio, evitando el uso innecesario de excepciones para control de flujo.

9.3. Uso de `Result/Result<T>`

Implementa el patrón `Result`. Todos los métodos del servicio devuelven una instancia de `Result` o `Result<T>` que indica explícitamente si la operación fue exitosa (`IsSuccess`) o si falló (`IsFailure`), adjuntando la lista o el detalle de los errores producidos.

9.4. Tratamiento de errores

Garantiza un control de errores predecible y limpio. Las excepciones no controladas se capturan en la capa superior o mediante logging, transformándolas en respuestas controladas de tipo `Result.Failure`.

10. Sincronización y notificaciones

10.1. Sincronización automática

Permite que el estado local de la aplicación vuelva a sincronizarse periódicamente con el estado expuesto por la API externa.

10.2. Funcionamiento de SynchroService

El flujo de ejecución corregido de SyncUsersAsync sigue estrictamente el siguiente orden:

1. Invoca `UserCache.Clear()` para vaciar por completo la caché.
2. Invoca `UserRepository.DeleteAllAsync()` para purgar la base de datos local.
3. Llama a `IJsonPlaceholderApi.GetUsersAsync()` para obtener los DTOs externos.
4. Recorre en un bucle la lista recibida: transforma cada DTO a Entidad y lo añade individualmente a la BBDD mediante `UserRepository.AddAsync(user)`.

10.3. PeriodicTimer

SynchroService utiliza `PeriodicTimer` de .NET dentro de un bucle asíncrono, permitiendo ejecutar la sincronización a intervalos regulares configurables sin bloquear hilos.

10.4. Notificaciones

Emite mensajes informativos ante operaciones CRUD o eventos de sincronización finalizados, registrando la actividad para auditoría.

11. Exportación y almacenamiento JSON

11.1. UserStorage

Servicio encargado de interactuar con el sistema de archivos físico del servidor para guardar o recuperar volcados de datos.

11.2. Serialización con System.Text.Json

Utiliza las APIs nativas de alta eficiencia de `System.Text.Json` configurando opciones como `WriteIndented = true` para formatear adecuadamente el archivo resultante.

11.3. Generación del archivo JSON

Crea la estructura de directorio si no existe y escribe el archivo JSON con la lista completa de usuarios formateada.

12. Logging

12.1. Uso de Serilog

Se integra Serilog como motor principal de registros estructurados reemplazando o extendiendo el proveedor por defecto de .NET.

12.2. Organización de los mensajes

Los mensajes incluyen plantillas enriquecidas con propiedades (por ejemplo: `_logger.Debug("[REPO-INIT] Base de datos inicializada correctamente")`), permitiendo filtrados y búsquedas avanzadas de logs.

13. Contenerización con Docker

13.1. Objetivo de la contenerización

Aislar la aplicación y sus dependencias en contenedores ligeros y portables para garantizar ejecuciones idénticas en cualquier entorno (desarrollo, testing, producción).

13.2. Docker Compose

Fichero `docker-compose.yml` que orquesta los servicios necesarios: la aplicación .NET, la base de datos PostgreSQL y el servidor de caché Redis.

13.3. PostgreSQL y Redis

- ✚ PostgreSQL: Servicio configurado sobre la imagen oficial postgres:alpine, especificando credenciales, nombre de base de datos y puerto 5432.
- ✚ Redis: Servicio sobre redis:alpine exponiendo el puerto 6379.

13.4. Volúmenes y Healthchecks

- ✚ Volúmenes: Montados para persistir los datos de PostgreSQL y Redis entre reinicios de contenedores.
- ✚ Healthchecks: Verificaciones de estado en PostgreSQL y Redis para asegurar que la aplicación .NET espere a que la infraestructura esté totalmente operativa antes de iniciar.

14. Pruebas

14.1. Proyecto de pruebas

Se dispone del proyecto RepositorioRemoto.Test, configurado como un proyecto de pruebas unitarias independiente dentro de la misma solución.

14.2. NUnit, Moq y FluentAssertions

- ✚ NUnit: Framework ejecutor de las pruebas unitarias ([TestFixture], [Test]).
- ✚ Moq: Herramienta para crear objetos simulados de las interfaces (Mock<IUserRepository>, Mock<IUserCache>, Mock<IJsonPlaceholderApi>).
- ✚ FluentAssertions: Biblioteca que proporciona una sintaxis expresiva y legible para las aserciones (ej: result.IsSuccess.Should().BeTrue()).

14.3. Resultados obtenidos

Todas las pruebas unitarias automatizadas se ejecutan con éxito, garantizando una alta cobertura del código (code coverage) y previniendo regresiones durante futuras refactorizaciones.

15. Decisiones de diseño

15.1. Uso de interfaces

Garantiza el principio de Inversión de Dependencias (DIP de SOLID). Permite acoplamiento débil entre clases y simplifica la sustitución de componentes y el testing.

15.2. Separación de responsabilidades

Cada clase tiene un único propósito claro (Single Responsibility Principle): los controladores/console gestionan la entrada, los servicios la lógica, los mappers la conversión, y los repositorios la persistencia.

15.3. Elección de SQLite y PostgreSQL

Se eligió SQLite por su facilidad de despliegue local en desarrollo sin requerir servicios externos, y PostgreSQL para entornos de producción contenerizados dada su robustez relacional.

15.4. Elección de MemoryCache y Redis

El soporte híbrido permite que la aplicación funcione de forma autónoma en máquinas sencillas (usando la RAM) o escala de forma distribuida en clústeres mediante Redis.

15.5. Uso de DTO y Mappers

Garantizan la protección de las entidades de dominio y permiten desacoplar el modelo de datos interno de los formatos de API de terceros.

15.6. Uso de Refit

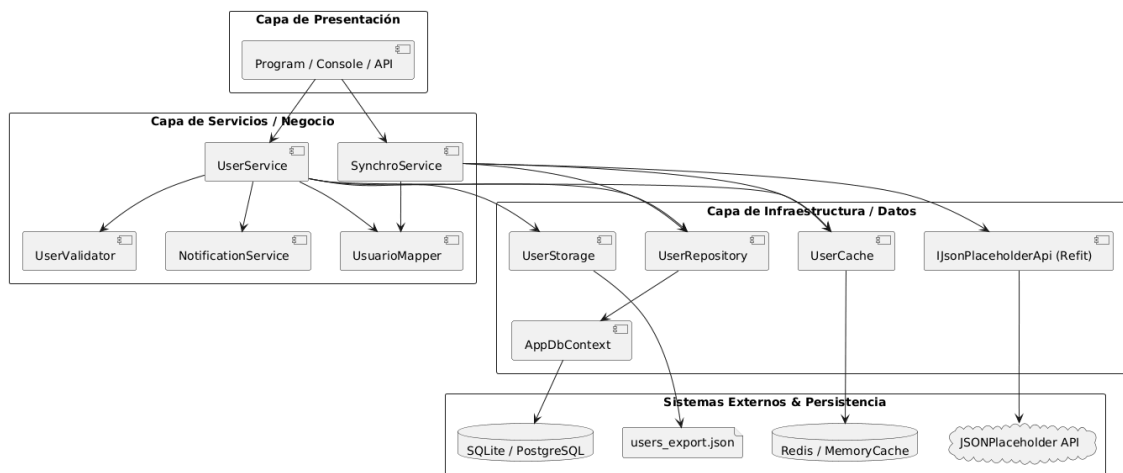
Su uso evita la escritura manual de boilerplate con HttpClient, reduciendo errores en las llamadas HTTP REST y centralizando los contratos en una interfaz fuertemente tipada.

15.7. Uso de Docker

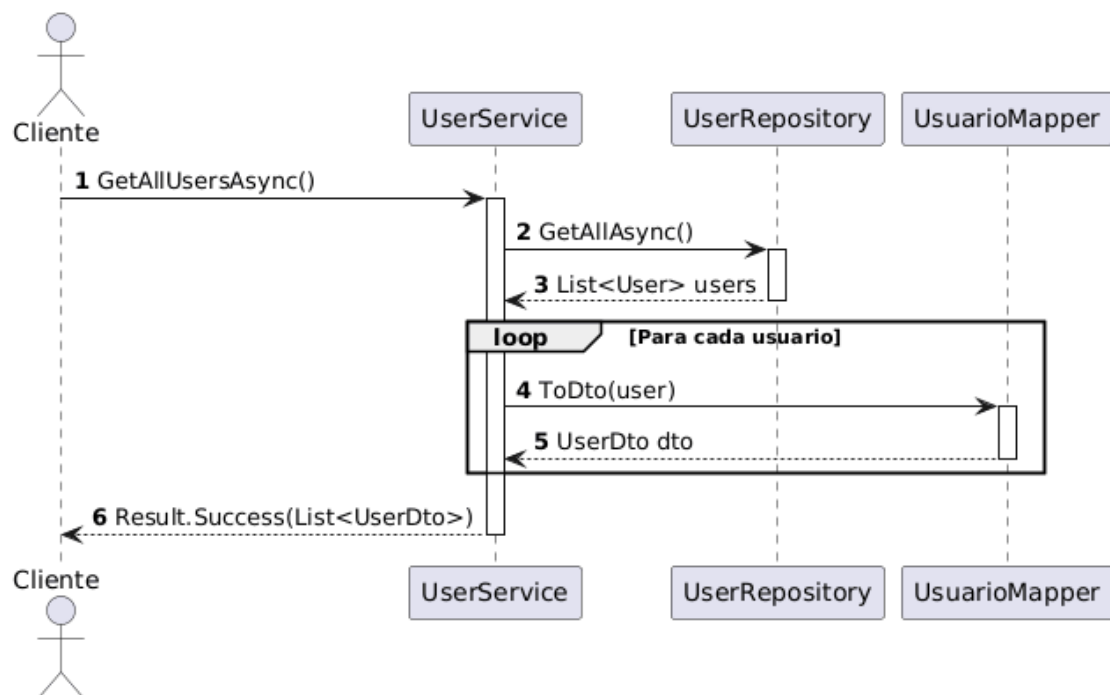
Elimina el problema de "funciona en mi máquina" (works on my machine), estandarizando el entorno de ejecución, librerías y dependencias de infraestructura en un único contenedor auto-contenido.

16. Diagramas y flujos de funcionamiento

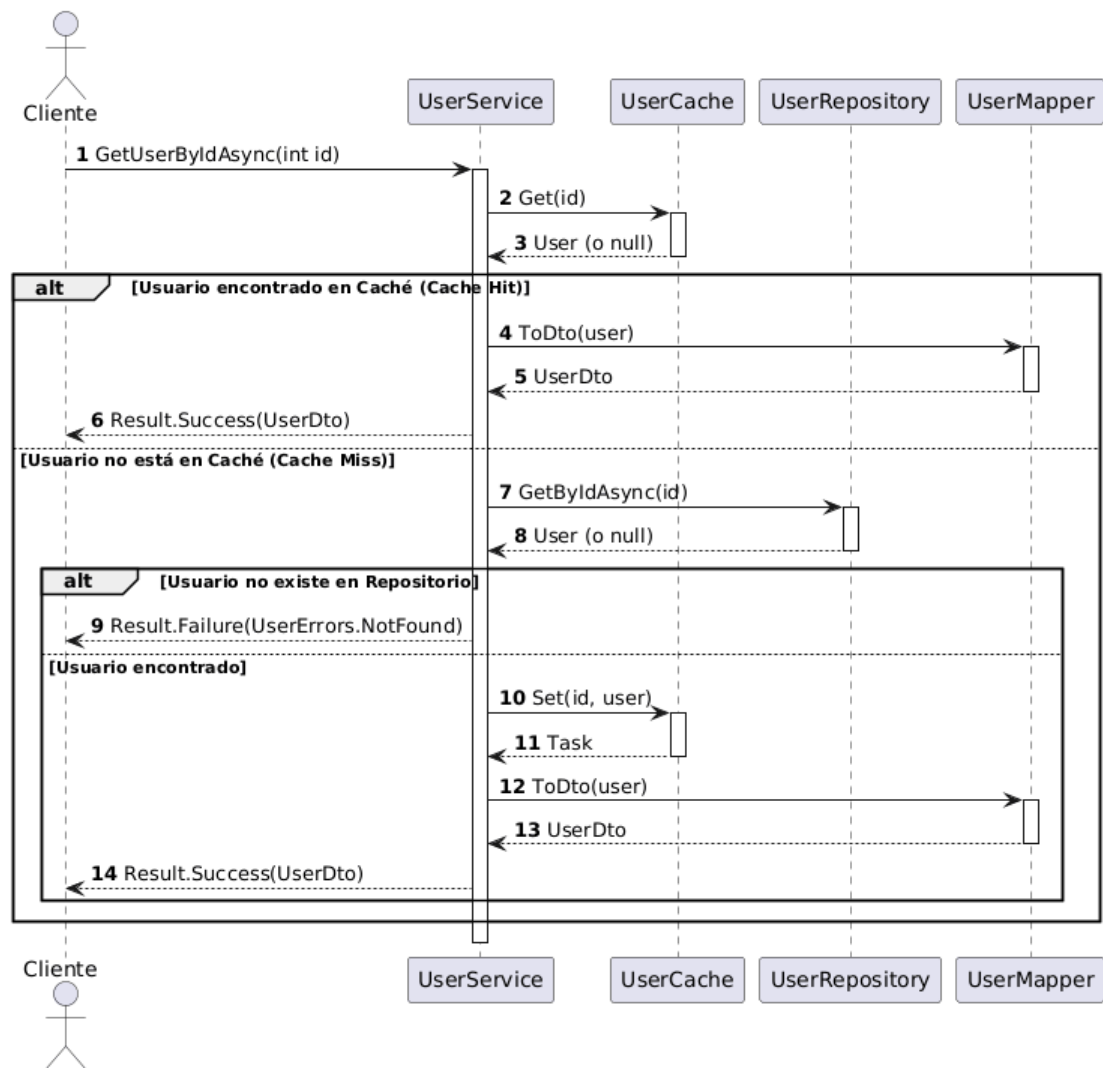
16.1. Diagrama de arquitectura



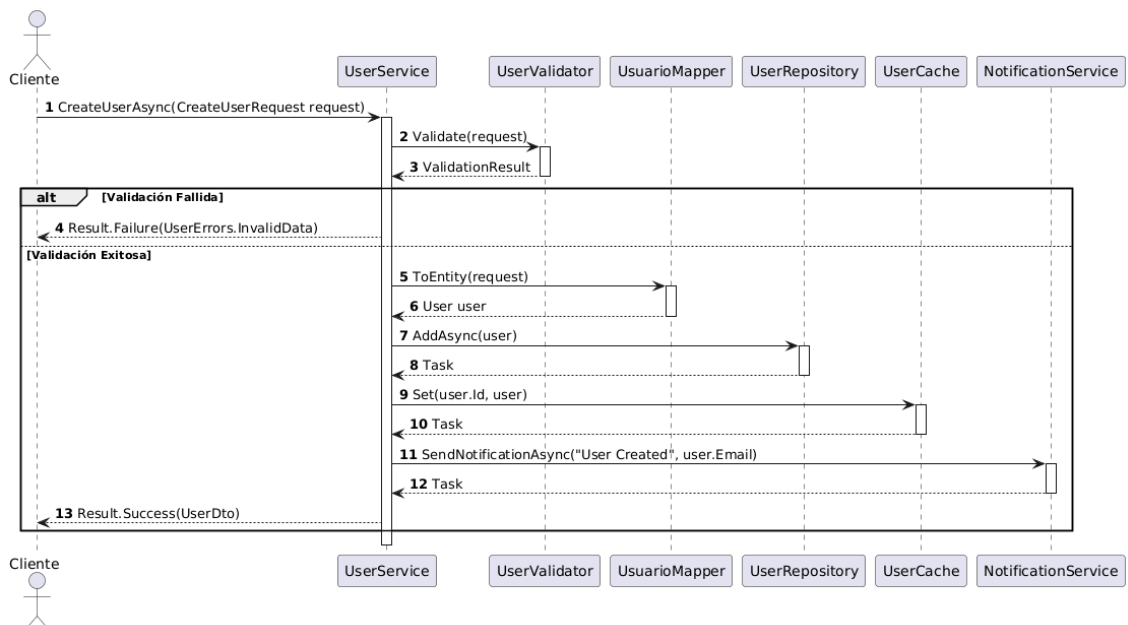
16.2. Flujo de obtención de usuarios



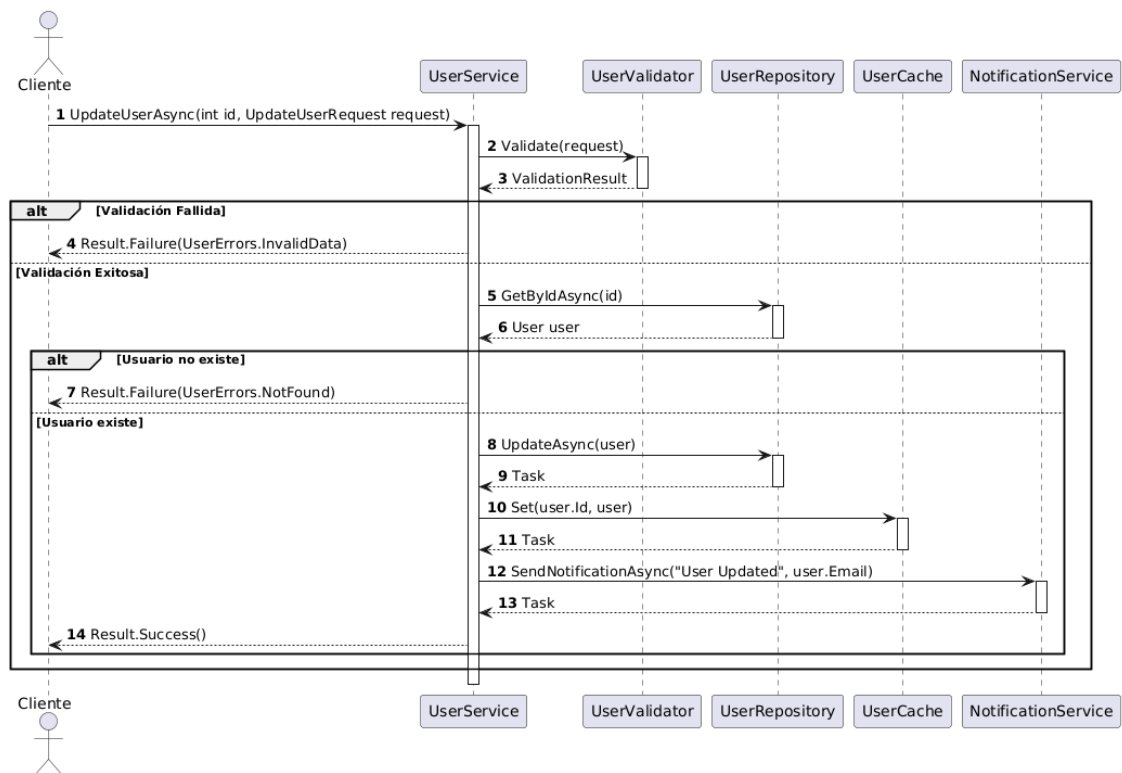
16.3. Flujo de consulta por ID



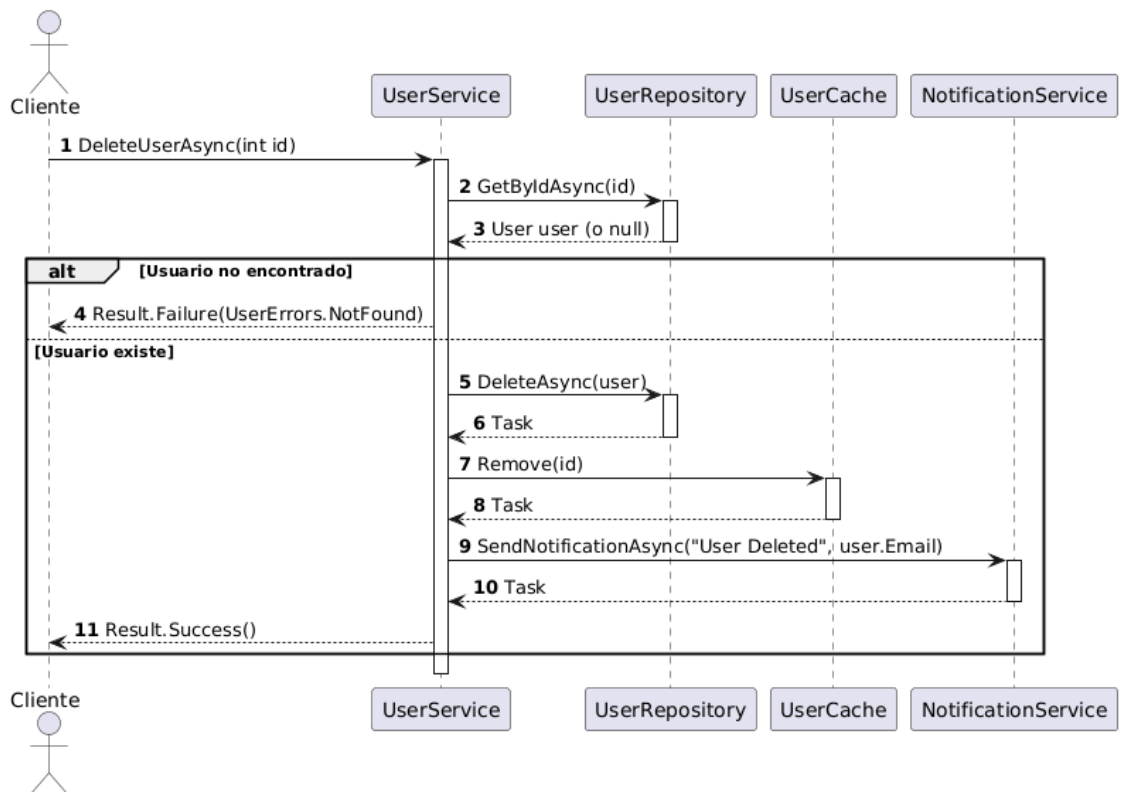
16.4. Flujo de creación



16.5. Flujo de actualización



16.6. Flujo de eliminación



16.7. Flujo de sincronización

